

Aufgabenblatt: Funktionen

Aufgabe 1: Bankautomat

Aufgabenstellung: Erstelle einen einfachen Bankautomaten. Das Programm soll über ein Menü gesteuert werden und folgende Funktionen enthalten:

- `show_balance(balance)` - zeigt den aktuellen Kontostand formatiert an
- `deposit()` - fragt nach einem Einzahlungsbetrag, prüft die Eingabe und gibt den Betrag zurück
- `withdraw(balance)` - fragt nach einem Auszahlungsbetrag, prüft die Eingabe und gibt den Betrag zurück

1. Das Hauptprogramm soll dem Nutzer wiederholt folgendes Menü anzeigen:

- i. Kontostand anzeigen
- ii. Einzahlung
- iii. Auszahlung
- iv. Beenden

2. Der Kontostand soll nach jeder Ein- und Auszahlung aktualisiert werden.

3. Ungültige Eingaben sollen abgefangen werden:

- Negative Beträge sind nicht erlaubt.
- Eine Auszahlung darf den Kontostand nicht übersteigen.

Beispielausgabe:

```
# # # # # # # #
#   BANK AUTOMAT   #
# # # # # # # #
1. Kontostand anzeigen
2. Einzahlung
3. Auszahlung
4. Beenden
Was möchten Sie tun?: 1
```

```
Hier ist ihr aktueller Kontostand:
# # # # # # # #
#   10.00 €       #
# # # # # # # #
```

Aufgabe 2: Einheitenumrechner

Beschreibung: Funktionen sind ideal um häufig benötigte Berechnungen zu kapseln und wiederverwendbar zu machen. Ein Einheitenumrechner ist ein klassisches Beispiel dafür.

Aufgabenstellung: Schreibe folgende Umrechnungsfunktionen. Jede Funktion soll den umgerechneten Wert zurückgeben, nicht ausgeben. Versehe jede Funktion mit einem Docstring.

- `km_zu_meilen(km)` - 1 km = 0.621371 Meilen
- `kg_zu_pfund(kg)` - 1 kg = 2.20462 Pfund
- `kmh_zu_ms(kmh)` - 1 km/h = 1/3.6 m/s
- `ms_zu_kmh(ms)` - Umkehrung der obigen Funktion

Schreibe danach ein Hauptprogramm, das den Benutzer nach einem Wert fragt und alle Umrechnungen ausgibt.

Beispielausgabe:

```
Wert eingeben: 100
```

```
100 km      = 62.14 Meilen
100 kg       = 220.46 Pfund
100 km/h     = 27.78 m/s
100 m/s      = 360.00 km/h
```

Aufgabe 3: Notenrechner

Beschreibung: Komplexere Programme lassen sich durch Funktionen in übersichtliche Teilaufgaben gliedern. Diese Aufgabe zeigt wie Funktionen zusammenarbeiten um ein größeres Programm zu strukturieren.

Aufgabenstellung: Schreibe ein Notenauswertungsprogramm mit folgenden Funktionen:

- `note_zu_text(note)` - gibt die Bezeichnung zurück (1 = Sehr gut, 2 = Gut, 3 = Befriedigend, 4 = Ausreichend, 5 = Mangelhaft, 6 = Ungenuegend)
- `durchschnitt(noten)` - berechnet den Notendurchschnitt
- `beste_note(noten)` - gibt die beste Note zurück
- `schlechteste_note(noten)` - gibt die schlechteste Note zurück
- `zeugnis_ausgeben(name, noten)` - ruft alle obigen Funktionen auf und gibt ein formatiertes Zeugnis aus

Das Hauptprogramm fragt nach einem Namen und 5 Noten und ruft dann `zeugnis_ausgeben()` auf.

Beispielausgabe:

```
Name: Maria
Note 1: 2
Note 2: 1
Note 3: 3
Note 4: 2
Note 5: 1
```

```
=====
Zeugnis für Maria
=====
Noten:      [2, 1, 3, 2, 1]
Durchschnitt: 1.80 (Gut)
```

Beste Note: 1 (Sehr gut)
Schlechteste: 3 (Befriedigend)
=====

Aufgabe 4: Speichereinheiten-Umrechner

Beschreibung: In der Informatik gibt es zwei verschiedene Systeme für Speichereinheiten: das dezimale System (1000er-Basis) und das binäre System (1024er-Basis). Funktionen machen den Unterschied zwischen beiden Systemen direkt sichtbar.

Aufgabenstellung: Schreibe Umrechnungsfunktionen für beide Systeme. Gehe jeweils von Byte als Ausgangsgröße aus.

Dezimales System (SI):

- `byte_zu_kb(byte)` - 1 KB = 1000 Byte
- `byte_zu_mb(byte)` - 1 MB = 1000² Byte
- `byte_zu_gb(byte)` - 1 GB = 1000³ Byte

Binäres System (IEC):

- `byte_zu_kib(byte)` - 1 KiB = 1024 Byte
- `byte_zu_mib(byte)` - 1 MiB = 1024² Byte
- `byte_zu_gib(byte)` - 1 GiB = 1024³ Byte

Schreibe ein Hauptprogramm, das eine Bytezahl einliest und beide Systeme gegenüberstellt.

Beispielausgabe:

Byte eingeben: 1000000000

Dezimales System (SI):

1000000000 Byte = 1000000.00 KB
1000000000 Byte = 1000.00 MB
1000000000 Byte = 1.00 GB

Binäres System (IEC):

1000000000 Byte = 976562.50 KiB
1000000000 Byte = 953.67 MiB
1000000000 Byte = 0.93 GiB

Aufgabe 5: Passwort-Generator

Beschreibung: Bisher hast du Passwörter geprüft und Login-Systeme gebaut. Jetzt dreht sich der Spieß um: Das Programm generiert selbst sichere Passwörter nach vorgegebenen Regeln.

Aufgabenstellung: Schreibe folgende Funktionen für einen Passwort-Generator:

- `generiere_passwort(laenge, grossbuchstaben=True, ziffern=True, sonderzeichen=True)`
 - baut aus den erlaubten Zeichenmengen ein zufälliges Passwort der gewünschten Länge und gibt es als String zurück
- `pruefe_staeke(password)` - bewertet das Passwort und gibt eine Einschätzung zurück: "schwach" , "mittel" oder "stark"
- `generiere_mehrere(anzahl, laenge)` - gibt eine Liste mit mehreren generierten Passwörtern zurück

Das Hauptprogramm fragt den Benutzer nach gewünschter Länge und Anzahl der Vorschläge und gibt diese mit ihrer Stärke aus.

Beispielausgabe:

```
Passwortlänge: 12
Anzahl Vorschläge: 3
```

```
Vorschlag 1: kR3#mZpA1!Bx  (stark)
Vorschlag 2: abcdefghijkl  (schwach)
Vorschlag 3: Bx9@QnLe2Tmw  (stark)
```

[Bonus] Taschenrechner mit Funktionen

Beschreibung: Der rote Faden des Kurses: Der Taschenrechner wurde in Einheit 06 einmalig gebaut, in Einheit 07 mit einer Schleife erweitert und in Einheit 08 um einen Verlauf ergänzt. Jetzt wird er mit Funktionen sauber strukturiert.

Aufgabenstellung: Strukturiere den Taschenrechner mit folgenden Funktionen:

- `berechne(a, op, b)` - führt die Berechnung durch und gibt das Ergebnis zurück; gibt `None` zurück bei ungültiger Operation oder Division durch 0
- `verlauf_ausgeben(verlauf)` - gibt alle bisherigen Berechnungen nummeriert aus
- `hauptprogramm()` - enthält die Hauptschleife mit Menü

Das Menü soll folgende Optionen bieten:

- `r` - Rechnen (mit Option das letzte Ergebnis zu übernehmen)
- `v` - Verlauf anzeigen
- `q` - Beenden

Beispielausgabe:

```
=== Taschenrechner ===
[r] Rechnen [v] Verlauf [q] Beenden

Auswahl: r
Erste Zahl: 10
Operation: *
Zweite Zahl: 5
```

Ergebnis: $10.0 * 5.0 = 50.0$

Auswahl: r

Mit Ergebnis weiterrechnen? (ja/nein): ja

Erste Zahl (übernommen): 50.0

Operation: /

Zweite Zahl: 4

Ergebnis: $50.0 / 4.0 = 12.5$

Auswahl: v

1. $10.0 * 5.0 = 50.0$

2. $50.0 / 4.0 = 12.5$

Auswahl: q

Auf Wiedersehen!